Dependency Injection
Mehr als nur ASP.NET

Olaf Lischke

Speaker: Olaf Lischke

kontaktdaten.vcf

macht .NET, seit es .NET gibt

versucht, Projekte und Seminare zu ko

singt Tenor in Chören und Musikprojekten

zockte schon auf dem ZX 81, heute ausschließlich auf PC

fotografiert, seit er eine Kamera halten kann

fliegt, wenn Wetter und Zeit es zulassen (TMG/SEP)

Dependency Injection Grundlagen

Was ist Dependency Injection?

Design-Pattern zur Umsetzung von Inversion of Control

- Klassen erzeugen ihre Abhängigkeiten nicht selbst
- Abhängigkeiten werden von außen bereitgestellt (oft, aber nicht ausschließlich per Constructor Injection)

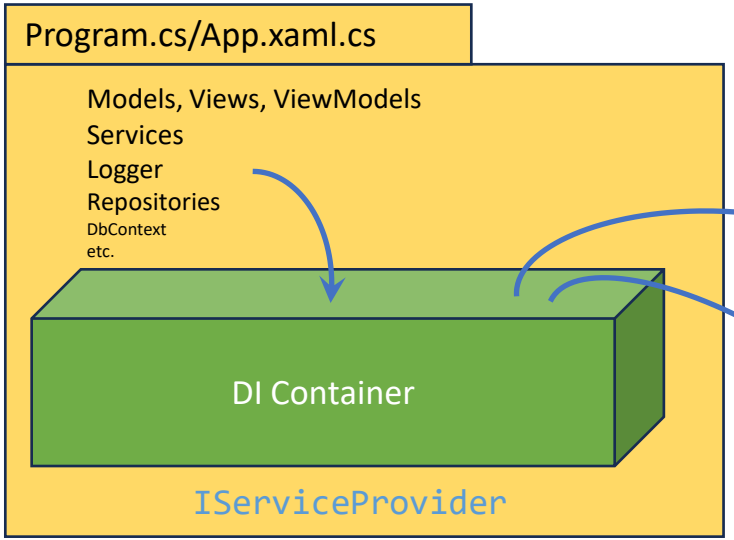
Ziel

- Entkopplung von Schnittstelle und Implementierung

Vorteile: Testbarkeit, Wartbarkeit, Flexibilität, SOLID-Prinzipien

Dependency Injection

nuget:
`Microsoft.Extensions.DependencyInjection`



Debug-Tipp:
Interface!

Constructor Injection

```
public MainWindowViewModel(IMessenger messenger, ILogger<Main
```

Instanzen automatisch aus DI Container

GetService-Methode

```
viewModel = (ViewModel)App.Current.Services
                                    .GetService(typeof(ViewMod
```

- instanziiert Komponenten auf Anfrage
- weiß, wie diese zu instanziieren sind
- kennt und kontrolliert die Lebensdauer

Zeit für ne **DEMO**

Scope Lifetime

- **Transient:** Neue Instanz bei jedem Resolve
- **Scoped:** Eine Instanz pro Scope (z.B. HTTP-Request)
- **Singleton:** Eine Instanz für gesamte Anwendung

Microsoft.Extensions.DependencyInjection

`Microsoft.Extensions.DependencyInjection`
Standard-DI-Container seit .NET Core

Überblick:

- Integriert in ASP.NET Core, Blazor, MAUI
- Über eine Milliarde Downloads von nuget.org
- Leichtgewichtig und performant

Key Features:

- Constructor Injection (Standard)
- Keyed Services ab .NET 8
- Service Validation ab .NET 8
- Native AOT:
  Volle Unterstützung ab .NET 10

Einschränkungen:

- Kein natives Assembly-Scanning
- Keine Property Injection
- Keine Module oder Factories

AutoFac: Etablierter DI-Container seit 2007

Überblick:

- Über 480 Millionen NuGet-Downloads
- 4.500+ GitHub Stars
- Fluent API und Module-System
- Großes Ecosystem (MediatR, MassTransit, etc.)

Stärken:

- Property und Method Injection
- Delegate Factories (Func, Owned)
- Assembly-Scanning und Module
- Decorator-Pattern-Unterstützung

Schwächen:

- Nicht AOT-kompatibel in .NET 10
- Höhere Komplexität und Lernkurve
- Community-Support statt Microsoft

Lamar:

Nachfolger von StructureMap, performance-fokussiert

Überblick:

- 10-15 Millionen NuGet-Downloads
- Etwa 600 GitHub Stars
- Registry-DSL für Konventionen
- Codegenerierung mit Roslyn

Stärken:

- Höchste Performance durch Codegenerierung
- Implementiert MS DI-Abstraktionen direkt
- Konventionsbasierte Registrierung
- Bessere AOT-Unterstützung als Autofac

Schwächen:

- Kleineres Ecosystem als Autofac
- Weniger Community-Support
- Registry-DSL weniger intuitiv

Vergleich: Verbreitung und Performance

NuGet-Downloads (Stand Feb 2026)

Container	Downloads	Marktanteil
MS DI	Milliarden	>80% (Standard)
Autofac	480 Millionen	~10-20%
Lamar	10-15 Millionen	<5%

Performance-Ranking

Platz	Container	Grund
🥇	**Lamar**	Roslyn-Codegenerierung, Inlining
🥈	**MS DI**	Optimiert für Standard-Szenarien
🥉	**Autofac**	Etwas höherer Overhead durch Features

Für die meisten Anwendungen sind alle drei performant genug.

Erweiterte Features im Überblick

Feature	MS DI	Autofac	Lamar
Property Injection	❌	✅	✅
Module/Registry	❌	✅	✅
Assembly-Scanning	❌	✅	✅
Delegate Factories	Manuell	✅	✅
Decorator	❌	✅	❌
Interceptors	❌	✅	✅
Keyed Services	✅ (.NET 8+)	✅	✅

AOT*-Kompatibilität

Container	AOT-Status	Details
MS DI	✅ Volle Unterstützung	Offiziell von Microsoft
Lamar	✅ Gute Unterstützung	Besonders in ASP.NET Core
Autofac	❌ Nicht kompatibel	Reflection-Probleme

***AOT? Ahead of Time Kompilierung! - Wichtig für:**

- Serverless-Szenarien (schneller Kaltstart)
- CLI-Tools (weil kleine Binaries)
- Edge-Computing (geringe Ressourcen)

Lernkurve

Container	Lernkurve	Grund
MS DI	🟢 Niedrig	Intuitiv, wenige Konzepte
Lamar	🟡 Mittel	Registry-Stil gewöhnungsbedürftig
Autofac	🔴 Hoch	Viele Features, Module, Factories

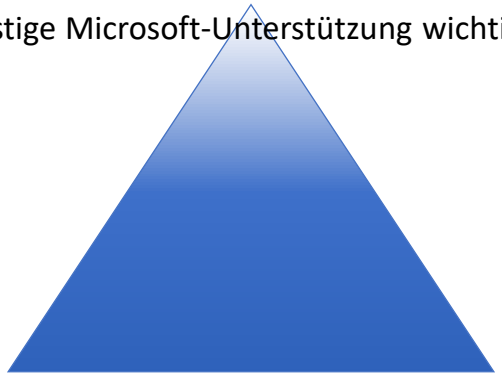
Support und Wartung

Container	Support-Typ	Langfristigkeit
MS DI	Microsoft	✅ Garantiert
Autofac	Community	✅ Etabliert seit 2007
Lamar	Community	⚠️ Kleineres Team

Wann welchen Container verwenden?

Microsoft DI (90% aller Projekte)

- Standard ASP.NET Core, Blazor, WPF/MAUI Apps
- Native AOT-Anforderungen
- Einfache bis mittlere Komplexität
- Langfristige Microsoft-Unterstützung wichtig



Autofac (Enterprise-Szenarien)

- Komplexe Enterprise-Anwendungen mit vielen Modulen
- Property/Method Injection benötigt
- Delegate Factories und Decorator-Pattern
- Viele Third-Party-Integrationen
- **Kein Native AOT geplant**

Lamar (Performance-kritisch)

- High-Performance Server-Anwendungen
- Konventionsbasierte Registrierung wichtig
- JasperFx/Marten/Wolverine Ecosystem
- Bessere AOT-Unterstützung als Autofac

„Start with MS DI, switch only when needed"

Faustregel: Bleib bei MS DI, bis es einen klaren Grund zum Wechsel gibt.

Einstieg mit Microsoft DI

- Deckt die allermeisten Szenarien ab
- Einfach, performant, zukunftssicher
- Keine externen Dependencies

Wechseln bei konkretem Bedarf

- **Autofac:** Feature-Reichtum und breites Ecosystem
- **Lamar:** Wenn Performance kritisch ist, AOT oder konventionsbasierte Registrierung benötigt wird

Ressourcen und Quellen:

- Microsoft DI:
learn.microsoft.com/dotnet/core/extensions/dependency-injection

- Autofac:
autofac.org

- Lamar:
jasperfx.github.io/lamar

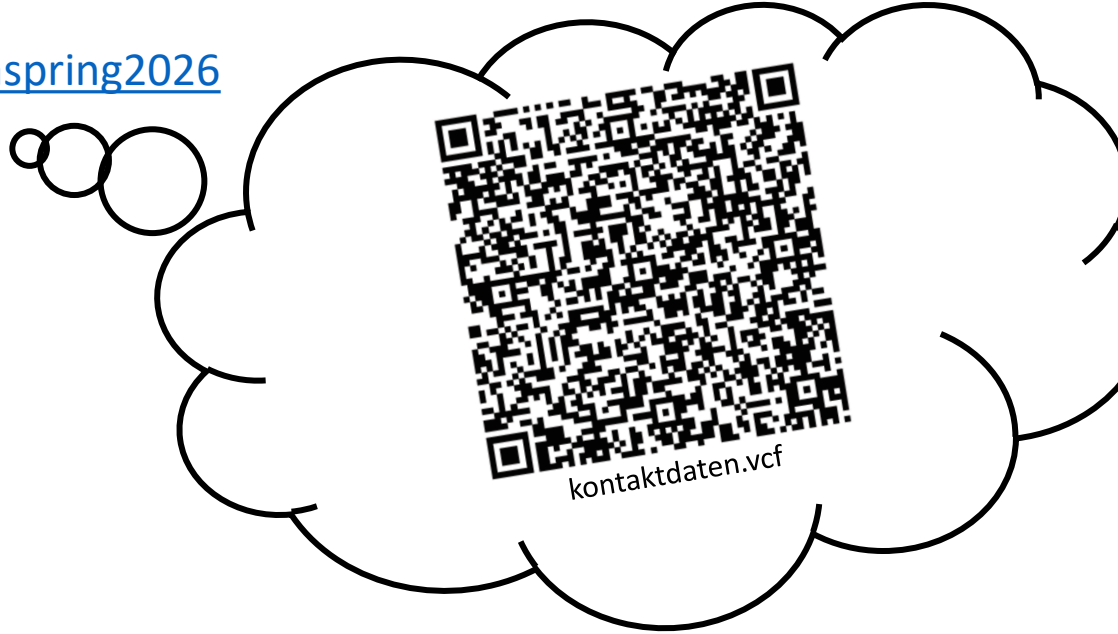
- Performance:
github.com/danielpalme/IocPerformance

Bitte

nicht vergessen

zu bewerten!

entwickler.de

Vielen Dank!

Slides und Code-Sample auf
https://github.com/olaflischke/bastaspring2026